The Scrypt library generates a unique random salt value for each password hash. The hash string that gets stored in the database includes the parameters that were used when the hash was generated, as well as this random salt value. This ensures that if you can always recreate the same hash in future, even if you later change the parameters. Your Scrypt library will be able to read this value and decode the parameters when it verifies the hash, as we will see later.

We can then add a new route for registering a new user to your Main. Locate the Main.java file in your editor and add the following lines just under where you previously created the SpaceController object:

```
    var userController = new UserController(database);
    post("/users", userController::registerUser);
```

3.3.4 Authenticating users in Natter

To authenticate a user, you'll extract the username and password from the HTTP Basic authentication header, lookup the corresponding user in the database, and finally verify the password matches the hash stored for that user. Behind the scenes, the Scrypt library will extract the salt from the stored password hash, then hash the supplied password with the same salt and parameters, and then finally compare the hashed password with the stored hash. If they match, then the user must have presented the same password and so authentication succeeds, otherwise it fails.